

Chatbot_RAG_pipeline

February 7, 2026

```
[211]: import nltk
from nltk import word_tokenize
from nltk import sent_tokenize

from nltk import pos_tag
from nltk.tag.mapping import map_tag

import numpy as np

nltk.download('all', quiet=True)
```

[211]: True

```
[212]: def get_upos_tags(text: str):
        """
        Tokenizes the text and returns (word, UPOS) pairs.
        Example UPOS tags: NOUN, VERB, ADJ, DET, ADP, etc.
        """
        word_list = word_tokenize(text)
        treebank_tags = pos_tag(word_list)
        upos_tags = [(word, map_tag('en-ptb', 'universal', tag)) for word, tag in
        ↪treebank_tags]
        return upos_tags
```

```
[213]: # Install the Sentence Transformer library
!pip install --quiet --upgrade sentence-transformers

import os
import logging

# Suppress Hugging Face / Transformers logs
os.environ["TRANSFORMERS_VERBOSITY"] = "error" # only show errors

logging.getLogger("transformers").setLevel(logging.ERROR)
logging.getLogger("sentence_transformers").setLevel(logging.ERROR)

from sentence_transformers import SentenceTransformer, util
```

```
# Load model silently
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
```

Loading weights: 0%| | 0/103 [00:00<?, ?it/s]

```
[214]: def get_embedding(text):
        return model.encode(text, convert_to_tensor=True)

def find_text_similarity(text1, text2):
    return round(float(util.pytorch_cos_sim(get_embedding(text1),
    ↪get_embedding(text2))[0][0]), 4)

def find_embedding_similarity(emb1, emb2):
    return round(float(util.pytorch_cos_sim(emb1, emb2)[0][0]), 4)

[215]: def create_embedding_list(text_list):
        return [get_embedding(text) for text in text_list]

[216]: def find_answer(query, text_list, embedding_list):

    threshold = 0.3
    if find_text_similarity(query, text_list[0]) < threshold:
        print('No answer found')
    else:
        print('Answer found')
    query_embedding = get_embedding(query)
    sim_list = [find_embedding_similarity(query_embedding, emb) for emb in
    ↪embedding_list]
    print("Similarity values between query and all sentences in the list:")
    print(sim_list)
    print("")
    return text_list[sim_list.index(max(sim_list))]

[217]: import requests
from bs4 import BeautifulSoup

def fetch_html(url: str) -> str:
    headers = {
        "User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/91.0.4472.114 Safari/537.36"
    }
    response = requests.get(url, headers=headers)
    response.raise_for_status()
    return response.text

def extract_paragraphs(html: str) -> list[str]:
```

```

soup = BeautifulSoup(html, "html.parser")

# Main content of Wikipedia pages is inside <div id="mw-content-text">
content_div = soup.find("div", {"id": "mw-content-text"})
paragraphs = []

if content_div:
    for p in content_div.find_all("p"):
        text = p.get_text().strip()
        if text: # ignore empty or citation-only paragraphs
            paragraphs.append(text)
    return paragraphs

```

```

[218]: def find_answer_top_k(query, text_list, embedding_list, k=2):
        query_embedding = get_embedding(query)
        sim_list = [find_embedding_similarity(query_embedding, emb) for emb in
        ↪embedding_list]

        # Get indices of top-k similarities (sorted descending)
        top_k_indices = np.argsort(sim_list)[::-1][:k]

        # Return top-k texts as a list
        return [text_list[i] for i in top_k_indices]

```

```

[219]: from openai import OpenAI

        from dotenv import load_dotenv
        import os
        load_dotenv("api_key.env")
        api_key = 'api_key'

        def get_ai_response(prompt):
            client = OpenAI(
                base_url="https://openrouter.ai/api/v1",
                api_key=api_key,
            )

            completion = client.chat.completions.create(
                model="tngtech/deepseek-r1t2-chimera:free",
                messages=[
                    {
                        "role": "user",
                        "content": prompt
                    }
                ]
            )

```

```
return completion.choices[0].message.content
```

RAG MODEL :

```
[220]: url = "https://en.wikipedia.org/wiki/Control_system"
html = fetch_html(url)
paragraphs = extract_paragraphs(html)
```

```
[221]: emb_list = create_embedding_list(paragraphs)
```

```
[222]: query = "Who are different types of control systems"
context_list = find_answer_top_k(query, paragraphs, emb_list, k=4)
for context in context_list:
    print(context)
    print("")
```

```
[223]: context = " ".join(context_list)

prompt = f"""

INSTRUCTIONS:
- Answer from the given context only.
- Think about the query carefully.

User Query: {query}.

Context: {context}
"""

response = get_ai_response(prompt)
print(response)
```

According to the provided context, the different types of control systems are classified based on their operation and mathematical representation:

```
**1. Open-Loop Control Systems**
- **Linear Control Systems**
  *Example:* Automatic toaster
- **Nonlinear Control Systems**
  *Example:* Automatic washing machine

**2. Closed-Loop Control Systems (Feedback Control Systems)**
- **Linear Control Systems**
  *Example:* Thermostat-controlled heating system
- **Nonlinear Control Systems**
  *Example:* Automatic missile launcher system
```

This classification is explicitly derived from the context.